

Embedded Firmware & Interfacing Documentation

Project Name: SignBoard Detection / HearSight Assistive Signboard Detection System

Platform: Raspberry Pi 5

Camera: Waveshare RPi IR-CUT Camera 3MP | **Interface:** 22-Pin CSI Connector (MIPI CSI-2)

Power Supply: 22.5W USB-C Power Bank

Programming Languages: Python 3, Shell Script (Bash)

Major Libraries: OpenCV · Picamera2 · Ultralytics YOLO · NumPy · gpiozero

Repository: <https://github.com/Ashok-19/SignBoard-Detection>

Documentation Date: June 2026

2. Project Overview

HearSight is a real-time assistive signboard detection and audio guidance system built on the Raspberry Pi 5. It is designed to help visually impaired users navigate public spaces by identifying signboards and announcing their content as voice alerts in English or Tamil.

- Captures a live video stream from the Waveshare RPi IR-CUT Camera 3MP connected via the 22-pin MIPI CSI-2 connector.
- Detects signboards using a YOLOv8 NCNN object detector — full-frame pass every N frames.
- Simultaneously runs a rolling tile inference sweep in a background thread to catch small or distant signs.
- Classifies each detected sign crop using a YOLOv8 NCNN classification model.
- Tracks detected signs across frames using a lightweight IoU+center-distance tracker.
- Decides the direction of the sign (Left / Ahead / Right) from bounding-box center X relative to frame width.
- Plays a directional audio alert via `ffplay`, choosing the correct language (English/Tamil) voice recording.
- Displays an annotated live preview window via `OpenCV imshow`.
- Launches automatically at login via `start_hearsight.sh` — handles audio (Bluetooth and wired) and `venv` activation.

3. Programming Languages Used

Language	Role in Project
Python 3.13	Main embedded firmware / inference logic. Camera, YOLO inference, tracking, audio, GPIO, display.
Shell Script (Bash)	Startup automation. venv activation, Bluetooth connection, Python script launch with production args.
OpenCV (cv2)	Frame rotation, display, bounding boxes, text overlays, preview window (cv2.imshow).

4. Hardware / Embedded Platform

Component	Description
Raspberry Pi 5	Main computing board. Runs Python inference, camera capture, audio, GPIO, and display. All YOLO inference runs on CPU using NCNN.
Waveshare RPi IR-CUT Camera 3MP	Connected via the 22-pin MIPI CSI-2 interface on RPi 5. Used for live video capture via Picamera2. NOT connected to GPIO pins.
22-Pin CSI Connector	Physical camera interface. The Raspberry Pi 5 uses a 22-pin flat-flex cable (FFC) connector for its CSI camera port.
22.5W USB-C Power Bank	Power supply. Provides adequate current for sustained CPU inference workloads on Raspberry Pi 5.
HDMI / Display Output	Monitor or screen connected via HDMI for the OpenCV preview window (cv2.imshow).
Audio Output (Bluetooth / Wired)	Supports both Bluetooth wireless and wired earphone/headphone output for voice announcements. Bluetooth handled via bluetoothctl.
CPU (Arm Cortex-A76, 4-core)	NCNN inference runtime. All YOLO models run on CPU — no GPU or dedicated accelerator.

5. Libraries Used

Versions detected at runtime on the Raspberry Pi 5 deployment environment.

Library / Package	Purpose in Project	Version	Required / Optional	Used In
Python	Interpreter and standard library	3.13.5	Required	Both
OpenCV (cv2)	Frame rotation, display, bounding boxes, text overlay, colour convert	4.10.0	Required	rpi_webcam_...py
NumPy	Array operations for frame and crop data	2.2.4	Required	rpi_webcam_...py
Picamera2	Raspberry Pi camera capture library (libcamera Python bindings)	version not exposed by pkg	Required	rpi_webcam_...py
Ultralytics YOLO	Load and run YOLOv8 NCNN detect and classify models	8.4.59	Required	rpi_webcam_...py
gpiozero	Optional GPIO control library (available in the Python environment)	not available from runtime	Optional	rpi_webcam_...py
argparse	CLI argument parsing (parse_args function)	stdlib	Required	rpi_webcam_...py
threading	FrameGrabber, TileWorker, InferenceWorker background threads	stdlib	Required	rpi_webcam_...py
subprocess	Launch ffmpeg for audio; check audio process status	stdlib	Required	rpi_webcam_...py
dataclasses	TwoStageDetection, TrackMemory, RuntimeState containers	stdlib	Required	rpi_webcam_...py
pathlib	Cross-platform file path handling for models and tuning files	stdlib	Required	rpi_webcam_...py
time	Monotonic clock for FPS, TTL, debounce, and track timing	stdlib	Required	rpi_webcam_...py
os	Environment variables, file listing, path checks	stdlib	Required	rpi_webcam_...py
sys	sys.argv pre-scan for --threads before heavy imports	stdlib	Required	rpi_webcam_...py
FFmpeg / ffmpeg	Audio playback of voice recording files (.mp4/.m4a/.mp3)	7.1.3-0+deb13u1+rpt1	Required	rpi_webcam_...py
bluetoothctl	Bluetooth device discovery and connection (CLI tool)	5.82	Required	start_hearsight.sh
Bash	Shell interpreter for the startup script	System Bash	Required	start_hearsight.sh

6. Pin Configuration & Physical Connections

IMPORTANT: The Waveshare RPi IR-CUT Camera 3MP is connected via the 22-pin MIPI CSI-2 flat-flex connector, NOT via GPIO pins. Both Bluetooth and wired audio output are supported. Bluetooth connection is handled by `bluetoothctl` in the shell script; wired earphones work immediately without any setup.

Interface / Pin	Function	Direction	Protocol / Mode	Component	Used In Code	Notes
22-Pin CSI Camera Port (FFC connector, RPi 5)	Camera frame input	Input	MIPI CSI-2	Waveshare RPi IR-CUT Camera 3MP	Picamera2 — <code>open_camera()</code> , <code>FrameGrabber</code>	NOT a GPIO pin. Uses dedicated 22-pin flat-flex cable interface.
HDMI Output	Preview display	Output	HDMI digital video	Monitor / display	<code>cv2.imshow(...)</code>	Requires a connected HDMI display.
Audio Output (BT / 3.5mm / USB)	Voice guidance audio	Output	BT A2DP / analog / USB audio	Bluetooth earphones or wired earphones	<code>subprocess.Popen(['ffplay', ...])</code>	Both Bluetooth and wired audio are supported. BT handled by <code>bluetoothctl</code> . Wired works immediately.
Bluetooth Adapter (integrated, RPi 5)	Wireless audio link	I/O	Bluetooth A2DP	Bluetooth earphones / headphones	<code>bluetoothctl</code> in <code>start_hearsight.sh</code>	Powered on via 'bluetoothctl power on'. 10-retry connect loop in background.
CPU (Cortex-A76)	NCNN model inference	Internal	NCNN CPU execution	YOLOv8 NCNN detector & classifier	<code>YOLO.predict(..., device='cpu')</code>	CPU-only inference. OMP/ OPENBLAS threads set from <code>--threads</code> before imports.
Storage / File System (SD card / SSD)	Model & audio file access	Input	File I/O	<code>weights/</code> and <code>voice_recording/</code> directories	<code>YOLO(args.detector, resolve_audio_file())</code>	Models must exist under <code>weights/</code> . Audio files under <code>voice_recording/</code> .
USB-C Power (22.5W power bank)	System power supply	Input	USB-C Power Delivery	22.5W power bank	N/A (hardware only)	Provides sufficient current for sustained CPU inference on RPi 5.

7. Firmware / Software Architecture

The firmware consists of two layers: the Bash startup script (`start_hearsight.sh`) which handles system initialisation, and the Python detection script (`rpi_webcam_test_two_stage_ncnn.py`) which runs the full AI inference pipeline.

7.1 Runtime Flow (Python Script)

Step 1 — Early Thread Configuration

`sys.argv` is scanned before any heavy imports to extract `--threads`. `OMP_NUM_THREADS` and `OPENBLAS_NUM_THREADS` are set so NumPy, OpenCV, and NCNN use the correct thread count.

Step 2 — Argument Parsing

`parse_args()` reads all CLI arguments. For preset selections (`accuracy/balanced/realtime`), defaults for model paths, image sizes, and TTLs are applied only where the user has not explicitly provided that argument.

Step 3 — Model and Camera Initialisation

Detector (YOLO detect) and classifier (YOLO classify) NCNN models are loaded. Picamera2 is opened with the selected resolution, FPS, and libcamera tuning file. A 1-second settle delay is applied after `camera.start()`.

Step 4 — Background Thread Start

FrameGrabber thread starts continuous camera frame capture. TileWorker thread starts the rolling tile inference sweep. InferenceWorker thread runs the full two-stage pipeline.

Step 5 — Main Display Loop

Each loop: reads frame → submits to TileWorker + InferenceWorker → retrieves detections → applies persist-TTL → updates tracks → evaluates audio candidates → draws preview → handles keyboard input.

Step 7 — Detection Pipeline

Full-frame detector runs every `--main-every` frames. Tile detections polled from TileWorker (non-blocking). Results merged with class-aware NMS. Depth/size gate filters outliers. Surviving crops are classified.

Step 8 — Tracking

`assign_track()` assigns each detection a stable ID using IoU+center-distance matching. TrackMemory stores timestamps, hit counts, and audio confirmation counts.

Step 9 — Audio Trigger Logic

Candidates filtered by: not yet spoken, fresh detection+classifier, `audio_stability` confirmations, `audio_debounce`, `audio_cls_gate` confidence. Highest-priority fires audio.

Step 10 — Direction and Audio File Resolution

Direction (left/ahead/right) from bounding-box center X relative to frame width thirds. `get_audio_dir()` maps class name to `voice_recording/` subfolder. `resolve_audio_file()` finds the matching language+direction file with multi-stage fuzzy filename matching.

Step 11 — Audio Playback

`ffplay` launched via `subprocess.Popen` with `-nodisp -autoexit` flags. Higher-priority signs preempt playing audio by terminating the subprocess.

Step 12 — Shutdown / Cleanup

On [q]: audio process terminated, `InferenceWorker` + `TileWorker` stopped, `FrameGrabber` released, camera stopped and closed, OpenCV windows destroyed.

8. Detailed File Documentation:

rpi_webcam_test_two_stage_ncnn.py

File location: testing/rpi_webcam_test_two_stage_ncnn.py

File size: 1591 lines | **Language:** Python 3

8.1 Section-by-Section Explanation

Imports and Environment Setup (Lines 1–46)

The script starts with a shebang and module docstring. Before importing heavy libraries, it scans `sys.argv` to extract `--threads` and sets `OMP_NUM_THREADS` and `OPENBLAS_NUM_THREADS`. Standard library imports follow, then external libraries (`cv2`, `numpy`, `picamera2`, `ultralytics YOLO`). `gpiozero` is imported in a try/except for systems without GPIO. `tile_inference` is imported with a relative/absolute fallback.

Model Path Constants (Lines 49–56)

`REPO_ROOT` is resolved from the script's own location using `Path(__file__).resolve().parents[1]`. Three preset model path pairs are defined: FAST (256px FP16), BALANCED (320px FP16), ACCURACY (640px). `DEFAULT_TUNING` points to the libcamera OV5647 NoIR tuning JSON.

Class-to-Audio Directory Mapping (Lines 58–83)

`PRETTY_CLASS_TO_AUDIO_DIR` maps each classifier class name to the corresponding audio folder name under `voice_recording/`. Handles naming mismatches between model output and folder names.

Class Priority Mapping (Lines 85–108)

`CLASS_PRIORITY` assigns tier 1 (Stop, No Entry, Danger Electricity), tier 2 (School Ahead, Railway Crossing, Speed Breaker), tier 3 (Bus Stop, Hospital, Toilet). Tier 1 preempts audio.

Audio Directory / File Resolution Functions (Lines 111–261)

`get_audio_dir()` maps a class name to a `voice_recording` subfolder with fuzzy fallback. `resolve_language_dir_and_files()` finds the correct language subdirectory (`eng/tam`). `resolve_audio_file()` selects the direction-specific file (`left/ahead/right`) with multi-stage filename matching (keyword → prefix → positional for 3-file sets).

Data Classes (Lines 264–342)

`TwoStageDetection`: result of one detected sign — track ID, box, detector/classifier names and confidences, freshness flags. `TrackMemory`: lifecycle of one tracked object — class ID, box, timestamps, hit count, audio confirmation state. `RuntimeState`: central mutable state — frame index, classifier cache, detection lists, audio process handle, language, track dictionaries, audio file cache.

Tracking Logic — `assign_track()` (Lines 344–404)

For each `RawDetection`, searches existing tracks of the same class. Computes composite score (`IoU×3 + center proximity + hit bonus`). Creates new track if no match qualifies.

Crop and Bounding-Box Helpers (Lines 408–469)

`crop_xyxy()`: padded image crop from frame, clamped to boundaries. Returns None if too small.
`clip_xyxy()`: clamps box to valid frame pixel ranges. `pretty_name()`: underscore-to-space conversion.

Depth / Size Gate — `passes_depth_gate()` (Lines 437–465)

Rejects detections too small (far away), too large (close background), or with extreme aspect ratio. All thresholds configurable via CLI args as fractions of frame size.

Camera Frame Grabbing — `FrameGrabber` class (Lines 496–533)

Background daemon thread calls `camera.capture_array('main')`, applies rotation (`cv2.rotate`), and stores latest frame under a Lock. `read()` returns latest frame without blocking the main loop.

Tile Worker — `TileWorker` class (Lines 536–711)

Background daemon thread iterates over tiles of the current frame (`iter_tiles`). Processes `--tile-budget` tiles per cycle. Results stored in per-tile cache; stale entries evicted after `--tile-cache-ttl`. Merged list published to `_latest_dets`. Main thread polls via `get_detections()`.

Inference Worker — `InferenceWorker` class (Lines 714–799)

Background daemon thread receives frames and calls `run_two_stage()` (detect + classify). Results stored under result lock, retrieved by main loop via `get_latest()`. Decouples heavy inference from the display/UI thread.

Full-Frame Detection (inside `run_two_stage`, Lines 878–897)

Full-frame detector runs every `--main-every` frames. Results extracted by `_extract_full_frame_raw()` and cached in `state.last_full_dets` until next refresh cycle.

Tile Detection (inside `run_two_stage`, Lines 899–915)

Tile detections read from `TileWorker.get_detections()` (non-blocking). Tile result TTL check (`result_ttl + 0.10s`) discards old results. Fresh results replace `state.last_tile_dets`.

Detection Merging (inside `run_two_stage`, Lines 917–940)

Full-frame + tile detections merged via `merge_detections_nms()` — class-aware NMS with full-frame priority. Merged results pass through depth/size/aspect gate.

Classification (inside `run_two_stage`, Lines 942–1010)

For each detection: track assigned, crop extracted, classifier run (single-crop sequential calls to avoid NCNN batch>1 IndexError bug). Results update `cls_cache`. `TwoStageDetection` objects assembled — accepted if `class != 'not_target'` and `conf >= --cls-conf`.

Display Drawing — `draw_detections()` (Lines 1016–1046)

Green (accepted) / grey (rejected) bounding rectangles with filled label backgrounds showing track ID, classifier name+confidence, detector confidence.

Camera Opening — open_camera() (Lines 1050–1078)

Loads libcamera tuning JSON, creates Picamera2, configures video stream with resolution, FPS, buffer_count=4, and optional camera controls (Sharpness, Contrast, etc.).

CLI Argument Parsing — parse_args() (Lines 1081–1249)

All CLI arguments defined via argparse. After initial parsing, provided() checks whether each arg was explicitly supplied. Presets apply defaults only for arguments not provided.

Runtime Main Loop — main() (Lines 1253–1587)

Initialises models, camera, state, threads. Main while True: reads frames, submits, retrieves detections, applies TTL, updates tracks, evaluates audio, draws preview, handles keyboard. try/finally ensures clean shutdown of all resources.

Audio Trigger Logic (Lines 1390–1492)

Per inference cycle: audio confirmations incremented for fresh matching detections. Candidates filtered by spoken status, freshness, stability, debounce, confidence gate. Highest-priority resolves direction+audio file, launches ffplay. Higher-priority signs terminate playing audio and start immediately.

Language Switching (Lines 1550–1556)

The [I] key switches audio language between English and Tamil at runtime without restarting the script.

Shutdown / Cleanup (Lines 1566–1587)

finally block: terminates ffplay process, stops InferenceWorker + TileWorker threads, releases FrameGrabber, stops and closes camera, destroys OpenCV windows. Final FPS summary printed.

8.2 Full Source Code — rpi_webcam_test_two_stage_ncnn.py

The complete, unmodified source code of the Python detection script follows in monospaced font.

```
#!/usr/bin/env python3
"""Raspberry Pi 5 live signboard detection using Picamera2 and NCNN models.

The detector keeps accuracy-oriented tiling, but schedules it as a rolling
background sweep instead of trying to finish every tile on every camera frame.
Full-frame proposals and classifier results are cached briefly for stable tracks.

Controls: [q] quit [p] pause [r] show/hide rejected proposals [f] fullscreen
"""

from __future__ import annotations

import sys
import os
import subprocess

# Quick pre-parse of --threads argument to set OMP/OPENBLAS threads before importing
cv2/numpy/ultralytics
threads = "4" # Default
for i, arg in enumerate(sys.argv):
    if arg == "--threads" and i + 1 < len(sys.argv):
        threads = sys.argv[i + 1]
        break
os.environ["OMP_NUM_THREADS"] = threads
os.environ["OPENBLAS_NUM_THREADS"] = threads

import argparse
import time
import threading
from dataclasses import dataclass, replace
from pathlib import Path

import cv2
import numpy as np
from picamera2 import Picamera2
from ultralytics import YOLO

try:
    from gpiozero import LED
    GPIO_SUPPORT = True
except (ImportError, Exception):
    GPIO_SUPPORT = False

try:
    from .tile_inference import RawDetection, TileRegion, _iou, iter_tiles, merge_detections_nms,
    preprocess_tile
except ImportError:
    from tile_inference import RawDetection, TileRegion, _iou, iter_tiles, merge_detections_nms,
    preprocess_tile
```

```

REPO_ROOT = Path(__file__).resolve().parents[1]
FAST_DETECTOR = REPO_ROOT / "weights/detector/best_ncnn_model_256_fp16"
FAST_CLASSIFIER = REPO_ROOT / "weights/classifier/best_ncnn_model_256_fp16"
BALANCED_DETECTOR = REPO_ROOT / "weights/detector/best_ncnn_model_320_fp16"
BALANCED_CLASSIFIER = REPO_ROOT / "weights/classifier/best_ncnn_model_320_fp16"
ACCURACY_DETECTOR = REPO_ROOT / "weights/detector/best_ncnn_model_640"
ACCURACY_CLASSIFIER = REPO_ROOT / "weights/classifier/best_ncnn_model_640"
DEFAULT_TUNING = Path("/usr/share/libcamera/ipa/rpi/pisp/ov5647_noir.json")

```

```

PRETTY_CLASS_TO_AUDIO_DIR = {
    "Accessible PwD": "accessible washroom",
    "Bus stop": "bus stop",
    "Cross road": "cross roads",
    "Danger electricity": "electricity danger",
    "Emergency exit": "emergency exit",
    "Fire alarm": "fire alarm",
    "Medical Shop - Hospital - First aid": "medical shop",
    "Men at work": "road work",
    "Narrow bridge ahead": "narrow road",
    "Narrow road ahead": "narrow road",
    "No entry": "no entry",
    "No parking": "no parking",
    "No stopping or Standing": "no stopping or standing",
    "Pedestrian Prohibited": "pedestrian prohibited",
    "Pedestrian crossing": "pedestrian crossing",
    "Public toilet": "public toilet",
    "Railway crossing": "railway crossing",
    "School ahead": "pedestrian crossing",
    "Side road left": "side road",
    "Side road right": "side road",
    "Speed breaker": "speed breaker",
    "Stop": "stop sign",
    "Washroom Female": "washroom female",
    "Washroom Male": "washroom male",
}

```

```

CLASS_PRIORITY = {
    "Stop": 1,
    "No entry": 1,
    "Danger electricity": 1,
    "No stopping or Standing": 1,
    "No parking": 1,
    "Pedestrian Prohibited": 1,
    "Railway crossing": 2,
    "School ahead": 2,
    "Pedestrian crossing": 2,
    "Speed breaker": 2,
    "Cross road": 2,
    "Narrow road ahead": 2,
    "Narrow bridge ahead": 2,
    "Men at work": 2,
}

```

```

"Side road left": 2,
"Side road right": 2,
"Accessible PwD": 3,
"Bus stop": 3,
"Medical Shop - Hospital - First aid": 3,
"Public toilet": 3,
"Washroom Female": 3,
"Washroom Male": 3,
}

def get_audio_dir(pretty_class_name: str) -> str | None:
    if pretty_class_name in PRETTY_CLASS_TO_AUDIO_DIR:
        return PRETTY_CLASS_TO_AUDIO_DIR[pretty_class_name]
    sanitized = pretty_class_name.lower().replace("-", " ").replace(" ", "").strip()
    try:
        folders = os.listdir("voice_recording")
        for folder in folders:
            folder_clean = folder.lower().strip()
            if sanitized == folder_clean or sanitized in folder_clean or folder_clean in sanitized:
                return folder
    except Exception:
        pass
    return None

def resolve_language_dir_and_files(voice_dir: str, language: str) -> tuple[str, list[str]]:
    base_dir = f"voice_recording/{voice_dir}"
    if not os.path.exists(base_dir):
        return None, []

    # Check for subdirectories first
    subdirs = []
    try:
        subdirs = [d for d in os.listdir(base_dir) if os.path.isdir(os.path.join(base_dir, d))]
    except Exception:
        pass

    # Match language subdirectories
    target_subdir = None
    if language == "tamil":
        for sd in subdirs:
            if sd.lower() in ["tam", "tamil"]:
                target_subdir = sd
                break
    else: # english
        for sd in subdirs:
            if sd.lower() in ["eng", "emg", "english"]:
                target_subdir = sd
                break

    if target_subdir:
        full_dir = os.path.join(base_dir, target_subdir)

```

```

        try:
            files = [f for f in os.listdir(full_dir) if f.endswith(".mp4") or f.endswith(".m4a") or
f.endswith(".mp3")]
            return full_dir, files
        except Exception:
            return full_dir, []

# If no subdirectory matches, look at the files directly in the base_dir
try:
    all_files = [f for f in os.listdir(base_dir) if os.path.isfile(os.path.join(base_dir, f)) and
(f.endswith(".mp4") or f.endswith(".m4a") or f.endswith(".mp3"))]
except Exception:
    all_files = []

# Filter files by language keywords
lang_files = []
for f in all_files:
    f_lower = f.lower()
    if language == "tamil":
        if any(k in f_lower for k in ["tam", "tamil"]):
            lang_files.append(f)
    else: # english
        if any(k in f_lower for k in ["eng", "emg", "english"]):
            lang_files.append(f)

# Process of elimination / fallback for direct files
if not lang_files and all_files:
    if len(all_files) == 2:
        other_lang = "english" if language == "tamil" else "tamil"
        other_files = []
        for f in all_files:
            f_lower = f.lower()
            if other_lang == "tamil" and any(k in f_lower for k in ["tam", "tamil"]):
                other_files.append(f)
            elif other_lang == "english" and any(k in f_lower for k in ["eng", "emg", "english"]):
                other_files.append(f)

        if len(other_files) == 1:
            remaining = [f for f in all_files if f not in other_files]
            if remaining:
                lang_files = remaining
        elif len(all_files) == 1:
            lang_files = all_files

    return base_dir, lang_files

def resolve_audio_file(voice_dir: str, language: str, direction: str) -> str | None:
    full_dir, files = resolve_language_dir_and_files(voice_dir, language)
    if not full_dir or not files:
        return None

    if len(files) == 1:

```

```

        return os.path.join(full_dir, files[0])

direction = direction.lower()

for file in files:
    f_clean = file.lower().strip()
    if direction == "left":
        if any(k in f_clean for k in ["left", "lft"]):
            return os.path.join(full_dir, file)
        if f_clean == "l.mp4" or f_clean == "l.m4a" or f_clean.startswith("l-") or
f_clean.startswith("le.") or f_clean.startswith("lt.")
            return os.path.join(full_dir, file)
        if any(k in f_clean for k in ["l", "-l-", "-l"]):
            return os.path.join(full_dir, file)
    elif direction == "right":
        if any(k in f_clean for k in ["right", "rgt", "rifht"]):
            return os.path.join(full_dir, file)
        if f_clean == "r.mp4" or f_clean == "r.m4a" or f_clean.startswith("r-") or
f_clean.startswith("re.") or f_clean.startswith("rt.")
            return os.path.join(full_dir, file)
        if any(k in f_clean for k in ["r", "-r-", "-r"]):
            return os.path.join(full_dir, file)
    else: # ahead
        if any(k in f_clean for k in ["ahead", "ahd", "agead"]):
            return os.path.join(full_dir, file)
        if f_clean == "a.mp4" or f_clean == "a.m4a" or f_clean.startswith("a-") or
f_clean.startswith("ae.") or f_clean.startswith("at.")
            return os.path.join(full_dir, file)
        if any(k in f_clean for k in ["a", "-ahead-", "-ahead"]):
            return os.path.join(full_dir, file)

for file in files:
    f_clean = file.lower().strip()
    first_char = f_clean[0] if len(f_clean) > 0 else ""
    if direction == "left" and first_char == "l":
        return os.path.join(full_dir, file)
    if direction == "right" and first_char == "r":
        return os.path.join(full_dir, file)
    if direction == "ahead" and first_char == "a":
        return os.path.join(full_dir, file)

if len(files) == 3:
    matched_files = {}
    for d in ["left", "right", "ahead"]:
        for file in files:
            f_clean = file.lower().strip()
            is_match = False
            if d == "left" and ("left" in f_clean or "lft" in f_clean or f_clean.startswith("l") or
"
l" in f_clean or "-l" in f_clean):
                is_match = True
            elif d == "right" and ("right" in f_clean or "rgt" in f_clean or "rifht" in f_clean or
f_clean.startswith("r") or " r" in f_clean or "-r" in f_clean):

```



```

        is_match = True
        elif d == "ahead" and ("ahead" in f_clean or "ahd" in f_clean or "agead" in f_clean or
f_clean.startswith("a") or " a" in f_clean or "-ahead" in f_clean):
            is_match = True
            if is_match:
                matched_files[d] = file
            if direction not in matched_files:
                other_files = [f for f in files if f not in matched_files.values()]
                if len(other_files) == 1:
                    return os.path.join(full_dir, other_files[0])

    return None

@dataclass
class TwoStageDetection:
    track_id: str | None
    box: tuple[int, int, int, int]
    detector_name: str
    detector_conf: float
    classifier_name: str
    classifier_conf: float
    accepted: bool
    fresh: bool = False
    classifier_fresh: bool = False

@dataclass
class TrackMemory:
    cls_id: int
    box: tuple[float, float, float, float]
    first_seen_time: float
    first_seen_frame: int
    last_seen_time: float
    last_seen_frame: int
    hits: int = 0
    audio_class_name: str | None = None
    audio_confirmations: int = 0
    last_audio_confirm_frame: int = 0

@dataclass
class RuntimeState:
    frame_index: int = 0
    cls_cache: dict[str, tuple[int, str, float]] | None = None
    last_full_dets: list[RawDetection] | None = None
    last_full_time: float = 0.0
    last_tile_dets: list[RawDetection] | None = None

    # Audio and language properties
    language: str = "english"
    audio_process: subprocess.Popen | None = None
    current_playing_priority: int = 999

```

```

track_first_seen: dict[str, tuple[float, int]] | None = None # track_id -> (time, frame_index)
spoken_tracks: set[str] | None = None
track_last_seen_frame: dict[str, int] | None = None
object_tracks: dict[str, TrackMemory] | None = None
audio_file_cache: dict[tuple[str, str, str], str | None] | None = None
last_tile_result_time: float = 0.0
_object_track_counter: int = 0

def __post_init__(self) -> None:
    if self.cls_cache is None:
        self.cls_cache = {}
    if self.last_full_dets is None:
        self.last_full_dets = []
    if self.last_tile_dets is None:
        self.last_tile_dets = []
    if self.track_first_seen is None:
        self.track_first_seen = {}
    if self.spoken_tracks is None:
        self.spoken_tracks = set()
    if self.track_last_seen_frame is None:
        self.track_last_seen_frame = {}
    if self.object_tracks is None:
        self.object_tracks = {}
    if self.audio_file_cache is None:
        self.audio_file_cache = {}

def purge_tracks(self, now: float, max_age: float) -> None:
    stale_ids = [
        tid for tid, track in self.object_tracks.items()
        if now - track.last_seen_time > max_age
    ]
    for tid in stale_ids:
        self.object_tracks.pop(tid, None)
        self.track_first_seen.pop(tid, None)
        self.track_last_seen_frame.pop(tid, None)
        self.spoken_tracks.discard(tid)
        if self.cls_cache:
            for key in list(self.cls_cache.keys()):
                if key.rsplit(":", 1)[-1] == tid:
                    self.cls_cache.pop(key, None)

def assign_track(
    self,
    raw: RawDetection,
    frame_w: int,
    frame_h: int,
    now: float,
    args: argparse.Namespace,
) -> str:
    """Lightweight detector-source-agnostic tracking for full and tile boxes."""
    max_age = max(0.15, float(getattr(args, "track_ttl_sec", 0.75)))
    self.purge_tracks(now, max_age)

```

```

x1, y1, x2, y2 = raw.xyxy
cx, cy = (x1 + x2) * 0.5, (y1 + y2) * 0.5
bw, bh = max(1.0, x2 - x1), max(1.0, y2 - y1)
cur_diag = float((bw * bw + bh * bh) ** 0.5)
frame_diag = float((frame_w * frame_w + frame_h * frame_h) ** 0.5)
iou_gate = max(0.05, float(getattr(args, "track_iou", 0.25)))
center_gate = max(0.02, float(getattr(args, "track_center_frac", 0.14)))

best_tid = None
best_score = -1.0
for tid, track in self.object_tracks.items():
    if track.cls_id != raw.cls_id:
        continue
    px1, py1, px2, py2 = track.box
    pcx, pcy = (px1 + px2) * 0.5, (py1 + py2) * 0.5
    prev_diag = float(((px2 - px1) ** 2 + (py2 - py1) ** 2) ** 0.5)
    center_dist = float(((cx - pcx) ** 2 + (cy - pcy) ** 2) ** 0.5)
    max_center_dist = max(frame_diag * center_gate, (cur_diag + prev_diag) * 0.9)
    cur_iou = _iou(raw.xyxy, track.box)
    if cur_iou < iou_gate and center_dist > max_center_dist:
        continue
    center_score = max(0.0, 1.0 - center_dist / max(max_center_dist, 1.0))
    score = cur_iou * 3.0 + center_score + min(track.hits, 5) * 0.01
    if score > best_score:
        best_tid = tid
        best_score = score

if best_tid is None:
    self._object_track_counter += 1
    best_tid = f"s{self._object_track_counter}"
    self.object_tracks[best_tid] = TrackMemory(
        cls_id=raw.cls_id,
        box=raw.xyxy,
        first_seen_time=now,
        first_seen_frame=self.frame_index,
        last_seen_time=now,
        last_seen_frame=self.frame_index,
        hits=0,
    )

track = self.object_tracks[best_tid]
track.box = raw.xyxy
if raw.fresh:
    track.last_seen_time = now
    track.last_seen_frame = self.frame_index
    track.hits += 1
    self.track_first_seen.setdefault(best_tid, (track.first_seen_time, track.first_seen_frame))
    self.track_last_seen_frame[best_tid] = self.frame_index
return best_tid

```

```

def crop_xyxy(frame: np.ndarray, box: tuple[float, float, float, float], pad: float, min_side: int) ->

```

```

np.ndarray | None:
height, width = frame.shape[:2]
x1, y1, x2, y2 = box
box_w = max(1.0, x2 - x1)
box_h = max(1.0, y2 - y1)
center_x = (x1 + x2) * 0.5
center_y = (y1 + y2) * 0.5
half_w = max(box_w * (0.5 + pad), min_side * 0.5)
half_h = max(box_h * (0.5 + pad), min_side * 0.5)
ix1 = max(0, int(center_x - half_w))
iy1 = max(0, int(center_y - half_h))
ix2 = min(width, int(center_x + half_w))
iy2 = min(height, int(center_y + half_h))
if ix2 <= ix1 + 2 or iy2 <= iy1 + 2:
    return None
return frame[iy1:iy2, ix1:ix2]

def clip_xyxy(frame: np.ndarray, box: tuple[float, float, float, float]) -> tuple[int, int, int, int]:
    height, width = frame.shape[:2]
    x1, y1, x2, y2 = box
    return (
        max(0, min(width - 1, int(x1))),
        max(0, min(height - 1, int(y1))),
        max(0, min(width, int(x2))),
        max(0, min(height, int(y2))),
    )

def passes_depth_gate(
    box: tuple[float, float, float, float],
    frame_w: int,
    frame_h: int,
    min_box_frac: float,
    max_box_frac: float,
    max_aspect_ratio: float,
) -> bool:
    """Reject detections that are too small (far away), too large (close / background),
    or have an extreme aspect ratio. All thresholds are relative to frame size."""
    x1, y1, x2, y2 = box
    bw = max(1.0, x2 - x1)
    bh = max(1.0, y2 - y1)

    # --- size gate (proxy for distance) ---
    frac_w = bw / frame_w
    frac_h = bh / frame_h
    if frac_w < min_box_frac or frac_h < min_box_frac:
        return False # too small / too far
    if frac_w > max_box_frac or frac_h > max_box_frac:
        return False # too large / too close

    # --- aspect ratio gate ---
    longer = max(bw, bh)

```

```

    shorter = min(bw, bh)
    if shorter > 0 and longer / shorter > max_aspect_ratio:
        return False # extreme sliver – unlikely a real sign

    return True

def pretty_name(name: str) -> str:
    return name.replace("__", " - ").replace("_", "-").replace("_", " ")

def classify_crops(
    classifier: YOLO,
    crops: list[np.ndarray],
    imgsz: int,
    device: str,
    infer_lock: threading.Lock | None = None,
) -> list:
    results = []
    for crop in crops:
        if infer_lock is None:
            res = classifier.predict(crop, imgsz=imgsz, device=device, verbose=False)
        else:
            with infer_lock:
                res = classifier.predict(crop, imgsz=imgsz, device=device, verbose=False)
        if res:
            results.append(res[0])
    return results

def should_refresh(cache_frame: int | None, frame_index: int, every: int) -> bool:
    return cache_frame is None or every <= 1 or (frame_index - cache_frame) >= every

class FrameGrabber:
    """Threaded camera capture for Picamera2. Decouples frame reading."""
    def __init__(self, camera: Picamera2, rotation: int = 0, camera_color: str = "rgb"):
        self._camera = camera
        self._rotation = rotation
        self._camera_color = camera_color
        self._frame: np.ndarray | None = None
        self._lock = threading.Lock()
        self._stopped = False
        self._thread = threading.Thread(target=self._grab_loop, daemon=True)
        self._thread.start()

    def _grab_loop(self) -> None:
        while not self._stopped:
            try:
                frame = self._camera.capture_array("main")
                if self._rotation == 90:
                    frame = cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)

```

```

        elif self._rotation == 180:
            frame = cv2.rotate(frame, cv2.ROTATE_180)
        elif self._rotation == 270:
            frame = cv2.rotate(frame, cv2.ROTATE_90_COUNTERCLOCKWISE)
        if self._camera_color == "rgb":
            frame = cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)

        with self._lock:
            self._frame = frame
    except Exception as e:
        print(f"[grabber] error: {e}")
        time.sleep(0.01)

def read(self) -> np.ndarray | None:
    with self._lock:
        return self._frame

def release(self) -> None:
    self._stopped = True
    self._thread.join(timeout=2.0)

class TileWorker:
    """Async rolling tile inference for NCNN on CPU.

    The laptop script can afford a full tiled sweep per submitted frame. On a
    Pi 5 CPU that starves the main loop. This worker processes a small tile
    budget per cycle, caches per-tile detections briefly, and continuously
    refreshes the sweep over the latest camera frame.
    """
    def __init__(self, detector_model: YOLO, args: argparse.Namespace, infer_lock: threading.Lock):
        self._args = args
        self._infer_lock = infer_lock
        self._frame: np.ndarray | None = None
        self._frame_shape: tuple[int, int] | None = None
        self._tiles: list[TileRegion] = []
        self._tile_index = 0
        self._frame_lock = threading.Lock()
        self._result_lock = threading.Lock()
        self._latest_dets: list[RawDetection] = []
        self._result_time: float = 0.0
        self._stopped = False
        self._tile_fps = 0.0
        self._tiles_per_sec = 0.0
        self._tile_cache: dict[tuple[int, int, int, int], tuple[float, list[RawDetection]]] = {}

        self._detector = detector_model
        self._thread = threading.Thread(target=self._run_loop, daemon=True)
        self._thread.start()

    def submit_frame(self, frame: np.ndarray) -> None:
        with self._frame_lock:
            self._frame = frame

```

```

def get_detections(self) -> tuple[list[RawDetection], float]:
    with self._result_lock:
        return list(self._latest_dets), self._result_time

@property
def tile_fps(self) -> float:
    return self._tile_fps

@property
def tile_count(self) -> int:
    return len(self._tiles)

@staticmethod
def _ordered_tiles(tiles: list[TileRegion], width: int, height: int, order: str) ->
list[TileRegion]:
    if order == "scan":
        return tiles
    focus_x = width * 0.5
    focus_y = height * 0.45
    return sorted(
        tiles,
        key=lambda tile: (
            (((tile.x1 + tile.x2) * 0.5 - focus_x) ** 2 + ((tile.y1 + tile.y2) * 0.5 - focus_y) **
2),
            tile.y1,
            tile.x1,
        ),
    )

def _run_loop(self) -> None:
    args = self._args
    tile_overlap = getattr(args, "tile_overlap", 0.30)
    tile_conf = getattr(args, "tile_conf", 0.25)
    tile_max_det = getattr(args, "max_det", 20)
    tile_min_box = getattr(args, "tile_min_box", 6)
    use_clahe = getattr(args, "tile_clahe", False)
    tile_budget = max(1, int(getattr(args, "tile_budget", 1)))
    tile_cache_ttl = max(0.1, float(getattr(args, "tile_cache_ttl", 0.75)))
    roi = getattr(args, "tile_roi", "full")
    scan_order = getattr(args, "tile_scan_order", "center")
    priority_every = max(0, int(getattr(args, "tile_priority_every", 0)))

    while not self._stopped:
        with self._frame_lock:
            frame = self._frame

        if frame is None:
            time.sleep(0.005)
            continue

        t0 = time.monotonic()
        height, width = frame.shape[:2]

```

```

        shape = (height, width)
        if shape != self._frame_shape or not self._tiles:
            tiles = iter_tiles(height, width, args.tile_size, tile_overlap, roi)
            self._tiles = self._ordered_tiles(tiles, width, height, scan_order)
            self._tile_index = 0
            self._frame_shape = shape
            self._tile_cache.clear()

        if not self._tiles:
            time.sleep(0.005)
            continue

        processed_tiles = 0
        processed_keys: set[tuple[int, int, int, int]] = set()
        for _ in range(min(tile_budget, len(self._tiles))):
            if self._stopped:
                break

            if priority_every and self._tile_index % priority_every == 0:
                tile = self._tiles[0]
                self._tile_index += 1
            else:
                tile = self._tiles[self._tile_index % len(self._tiles)]
                self._tile_index = (self._tile_index + 1) % len(self._tiles)

            crop = frame[tile.y1:tile.y2, tile.x1:tile.x2]
            if crop.shape[0] < 32 or crop.shape[1] < 32:
                continue

            tile_key = (tile.x1, tile.y1, tile.x2, tile.y2)
            processed_keys.add(tile_key)
            processed = preprocess_tile(crop, use_clahe=use_clahe)
            with self._infer_lock:
                res = self._detector.predict(
                    processed,
                    imgsiz=getattr(args, 'tile_imgsiz', args.det_imgsiz),
                    conf=tile_conf,
                    iou=args.det_iou,
                    device="cpu",
                    max_det=tile_max_det,
                    verbose=False,
                )

            processed_tiles += 1
            tile_dets: list[RawDetection] = []
            if not res or res[0].boxes is None:
                self._tile_cache[tile_key] = (time.monotonic(), tile_dets)
                continue

            tile_w = tile.x2 - tile.x1
            tile_h = tile.y2 - tile.y1

            for box in res[0].boxes:
                bx1, by1, bx2, by2 = box.xyxy[0].cpu().tolist()
                bw = bx2 - bx1
                bh = by2 - by1
                if bw < tile_min_box or bh < tile_min_box:
                    continue

```



```

        if bw > tile_w * 0.95 and bh > tile_h * 0.95:
            continue
        # Reject extreme aspect-ratio slivers (common tile false positive)
        longer, shorter = max(bw, bh), min(bw, bh)
        if shorter > 0 and longer / shorter > 6.0:
            continue
        cls_id = int(box.cls[0])
        tile_dets.append(
            RawDetection(
                xyxy=(bx1 + tile.x1, by1 + tile.y1, bx2 + tile.x1, by2 + tile.y1),
                conf=float(box.conf[0]),
                cls_id=cls_id,
                cls_name=res[0].names.get(cls_id, f"det_cls_{cls_id}"),
                source="tile",
            )
        )
        self._tile_cache[tile_key] = (time.monotonic(), tile_dets)

    now = time.monotonic()
    all_dets: list[RawDetection] = []
    stale_keys = []
    for key, (ts, dets) in self._tile_cache.items():
        if now - ts <= tile_cache_ttl:
            is_fresh = key in processed_keys
            all_dets.extend(replace(det, fresh=is_fresh) for det in dets)
        else:
            stale_keys.append(key)
    for key in stale_keys:
        self._tile_cache.pop(key, None)

    with self._result_lock:
        self._latest_dets = all_dets
        self._result_time = time.monotonic()

    elapsed = time.monotonic() - t0
    self._tile_fps = 1.0 / max(elapsed, 0.001)
    self._tiles_per_sec = processed_tiles / max(elapsed, 0.001)

    def stop(self) -> None:
        self._stopped = True
        self._thread.join(timeout=3.0)

class InferenceWorker:
    """Async full pipeline inference worker.

    The main UI loop only submits the latest frame and renders the newest
    available detections. This removes large FPS swings caused by synchronous
    full-frame detect + classify bursts on the display thread.
    """

    def __init__(
        self,

```

```

        detector: YOLO,
        classifier: YOLO,
        state: RuntimeState,
        args: argparse.Namespace,
        tile_worker: TileWorker | None,
        infer_lock: threading.Lock,
    ):
        self._detector = detector
        self._classifier = classifier
        self._state = state
        self._args = args
        self._tile_worker = tile_worker
        self._infer_lock = infer_lock
        self._frame: tuple[np.ndarray, float] | None = None
        self._frame_lock = threading.Lock()
        self._result_lock = threading.Lock()
        self._latest_detections: list[TwoStageDetection] = []
        self._latest_tile_count: int = 0
        self._result_time: float = 0.0
        self._result_frame_time: float = 0.0
        self._infer_fps: float = 0.0
        self._stopped = False
        self._thread = threading.Thread(target=self._run_loop, daemon=True)
        self._thread.start()

    def submit_frame(self, frame: np.ndarray) -> None:
        with self._frame_lock:
            self._frame = (frame, time.monotonic())

    def get_latest(self) -> tuple[list[TwoStageDetection], int, float, float]:
        with self._result_lock:
            return (
                list(self._latest_detections),
                self._latest_tile_count,
                self._result_time,
                self._result_frame_time,
            )

    @property
    def infer_fps(self) -> float:
        return self._infer_fps

    def _run_loop(self) -> None:
        while not self._stopped:
            with self._frame_lock:
                item = self._frame
                self._frame = None

            if item is None:
                time.sleep(0.002)
                continue

            frame, frame_time = item

```

```

        t0 = time.monotonic()
        detections, tile_count = run_two_stage(
            frame,
            self._detector,
            self._classifier,
            self._state,
            self._args,
            self._tile_worker,
            self._infer_lock,
        )
        t1 = time.monotonic()

        with self._result_lock:
            self._latest_detections = detections
            self._latest_tile_count = tile_count
            self._result_time = t1
            self._result_frame_time = frame_time

        self._infer_fps = 1.0 / max(t1 - t0, 0.001)

    def stop(self) -> None:
        self._stopped = True
        self._thread.join(timeout=3.0)

def run_ncnn_detector_on_tiles(
    detector: YOLO,
    frame: np.ndarray,
    tiles: list[TileRegion],
    args: argparse.Namespace,
) -> list[RawDetection]:
    """Run NCNN batch-1 inference and remap each tile to frame coordinates."""
    detections: list[RawDetection] = []
    for tile in tiles:
        crop = frame[tile.y1:tile.y2, tile.x1:tile.x2]
        crop = preprocess_tile(crop, use_clahe=args.tile_clahe)
        results = detector.predict(
            crop,
            imgsz=args.det_imgsz,
            conf=args.det_conf,
            iou=args.det_iou,
            device="cpu",
            max_det=args.max_det,
            verbose=False,
        )
        if not results or results[0].boxes is None:
            continue
        for box in results[0].boxes:
            x1, y1, x2, y2 = box.xyxy[0].cpu().tolist()
            if x2 - x1 < args.tile_min_box or y2 - y1 < args.tile_min_box:
                continue
            cls_id = int(box.cls[0])
            detections.append(

```

```

        RawDetection(
            xyxy=(x1 + tile.x1, y1 + tile.y1, x2 + tile.x1, y2 + tile.y1),
            conf=float(box.conf[0]),
            cls_id=cls_id,
            cls_name=results[0].names.get(cls_id, f"det_cls_{cls_id}"),
            source="tile",
        )
    )
    return detections

```

```

def _extract_full_frame_raw(det_result) -> list[RawDetection]:
    """Convert Ultralytics detection result to list of RawDetection."""
    raw: list[RawDetection] = []
    if det_result boxes is None:
        return raw
    for box in det_result.boxes:
        cls_id = int(box.cls[0])
        raw.append(RawDetection(
            xyxy=tuple(box.xyxy[0].cpu().tolist()),
            conf=float(box.conf[0]),
            cls_id=cls_id,
            cls_name=det_result.names.get(cls_id, f"det_cls_{cls_id}"),
            source="full",
        ))
    return raw

```

```

def run_two_stage(
    frame: np.ndarray,
    detector: YOLO,
    classifier: YOLO,
    state: RuntimeState,
    args: argparse.Namespace,
    tile_worker: TileWorker | None = None,
    infer_lock: threading.Lock | None = None,
) -> tuple[list[TwoStageDetection], int]:
    state.frame_index += 1
    now = time.monotonic()
    infer_kwargs = dict(
        imgsz=args.det_imgsz,
        conf=args.det_conf,
        iou=args.det_iou,
        device="cpu",
        max_det=args.max_det,
        verbose=False,
    )

    # --- Stage 1: full-frame detector proposals (periodic refresh) ---
    # NCNN full-frame inference is too expensive to block every display frame on
    # Pi 5. Keep the most recent full-frame proposals briefly and let the rolling
    # tile worker refresh small-object proposals continuously in the background.
    main_every = max(0, int(getattr(args, "main_every", 5)))

```

```

run_full = (
    args.zoom_mode != "tiles"
    and main_every > 0
    and (state.last_full_time <= 0.0 or state.frame_index % main_every == 1)
)
det_result = None
if run_full:
    if infer_lock is None:
        det_results = detector.predict(frame, **infer_kwargs)
    else:
        with infer_lock:
            det_results = detector.predict(frame, **infer_kwargs)
        det_result = det_results[0] if det_results else None
        state.last_full_dets = _extract_full_frame_raw(det_result) if det_result else []
        state.last_full_time = time.monotonic()

# --- Get tile detections from async worker (non-blocking) ---
tile_count = 0
if args.zoom_mode != "off" and tile_worker is not None:
    t_dets, t_time = tile_worker.get_detections()
    tile_age = time.monotonic() - t_time if t_time > 0 else 999.0
    tile_result_ttl = max(0.25, float(getattr(args, "result_ttl", 0.65)) + 0.10)
    if tile_age < tile_result_ttl:
        is_new_tile_result = t_time > 0 and t_time != state.last_tile_result_time
        state.last_tile_dets = (
            t_dets if is_new_tile_result
            else [replace(det, fresh=False) for det in t_dets]
        )
        if is_new_tile_result:
            state.last_tile_result_time = t_time
    else:
        state.last_tile_dets = []
    tile_count = tile_worker.tile_count

# --- Merge full-frame + tile detections ---
# Full-frame results persist until the next full-frame refresh replaces them.
# No TTL-based expiry: the periodic main_every cycle handles freshness.
full_raw = [replace(det, fresh=run_full) for det in state.last_full_dets]

if args.zoom_mode != "off" and state.last_tile_dets:
    merged_raw = merge_detections_nms(full_raw, state.last_tile_dets, iou_threshold=args.merge_iou)
else:
    merged_raw = full_raw

if not merged_raw:
    return [], tile_count

# --- Depth / size gate: reject too-small (far), too-large (close), extreme aspect ---
frame_h, frame_w = frame.shape[:2]
min_frac = getattr(args, "min_box_frac", 0.012)
max_frac = getattr(args, "max_box_frac", 0.55)
max_ar = getattr(args, "max_aspect_ratio", 6.0)
merged_raw = [

```

```

        r for r in merged_raw
        if passes_depth_gate(r.xyxy, frame_w, frame_h, min_frac, max_frac, max_ar)
    ]
    if not merged_raw:
        return [], tile_count

# --- Build metadata + classify ---
# Use one lightweight tracker for both full-frame and tile proposals. The
# Ultralytics tracker only sees intermittent full-frame detections and cannot
# maintain IDs for tile-only signs.
pending_cls: list[tuple[int, np.ndarray]] = []
metadata: list[dict] = []
cls_fresh_keys: set[str] = set()

for raw in merged_raw:
    xyxy = raw.xyxy
    crop = crop_xyxy(frame, xyxy, args.crop_pad, args.min_crop_side)
    if crop is None:
        continue

    det_name = raw.cls_name

    track_id = state.assign_track(raw, frame_w, frame_h, now, args)

    cache_key = f"{det_name}:{track_id}"

    cls_cached = state.cls_cache.get(cache_key)
    audio_verify = bool(getattr(args, "audio_reclassify_fresh", True)) and raw.fresh
    if cls_cached is None or audio_verify or should_refresh(cls_cached[0], state.frame_index,
args.classify_every):
        pending_cls.append((len(metadata), crop))

    metadata.append(
        {
            "track_id": track_id,
            "cache_key": cache_key,
            "box": clip_xyxy(frame, xyxy),
            "det_name": det_name,
            "det_conf": raw.conf,
            "fresh": raw.fresh,
        }
    )

if pending_cls:
    # Run sequentially on NCNN classifier to avoid the batch size > 1 IndexError bug
    cls_results = classify_crops(classifier, [c for _, c in pending_cls], args.cls_imgsz, "cpu",
infer_lock)
    for (meta_idx, _), cls_result in zip(pending_cls, cls_results):
        probs = cls_result.probs
        if probs is None:
            continue
        cls_id = int(probs.top1)
        cls_conf = float(probs.top1conf)

```

```

        cls_name = cls_result.names.get(cls_id, f"cls_{cls_id}")
        state.cls_cache[metadata[meta_idx]["cache_key"]] = (state.frame_index, cls_name, cls_conf)
        cls_fresh_keys.add(metadata[meta_idx]["cache_key"])

detections: list[TwoStageDetection] = []
for item in metadata:
    cls_cached = state.cls_cache.get(item["cache_key"])
    if cls_cached is None:
        continue
    _, cls_name, cls_conf = cls_cached
    accepted = cls_name != args.reject_class and cls_conf >= args.cls_conf
    detections.append(
        TwoStageDetection(
            track_id=item["track_id"],
            box=item["box"],
            detector_name=item["det_name"],
            detector_conf=item["det_conf"],
            classifier_name=pretty_name(cls_name),
            classifier_conf=cls_conf,
            accepted=accepted,
            fresh=bool(item["fresh"]),
            classifier_fresh=item["cache_key"] in cls_fresh_keys,
        )
    )

return detections, tile_count

def draw_detections(
    frame: np.ndarray,
    detections: list[TwoStageDetection],
    show_rejected: bool,
    line_width: int = 2,
    scale_x: float = 1.0,
    scale_y: float = 1.0,
    copy_frame: bool = True,
) -> np.ndarray:
    out = frame.copy() if copy_frame else frame
    for det in detections:
        if not det.accepted and not show_rejected:
            continue
        x1 = int(det.box[0] * scale_x)
        y1 = int(det.box[1] * scale_y)
        x2 = int(det.box[2] * scale_x)
        y2 = int(det.box[3] * scale_y)
        color = (0, 220, 0) if det.accepted else (90, 90, 90)
        cv2.rectangle(out, (x1, y1), (x2, y2), color, line_width)

        tid = f"#{det.track_id} " if det.track_id is not None else ""
        if det.accepted:
            label = f"{tid}{det.classifier_name} cls {det.classifier_conf:.2f} det
{det.detector_conf:.2f}"

```

```

        else:
            label = f"{tid}rejected: {det.classifier_name} {det.classifier_conf:.2f}"

            (tw, th), _ = cv2.getTextSize(label, cv2.FONT_HERSHEY_SIMPLEX, 0.45, 1)
            ly = y1 - 6 if y1 - th - 8 > 0 else y1 + th + 8
            cv2.rectangle(out, (x1, ly - th - 5), (x1 + tw + 6, ly + 3), color, -1)
            cv2.putText(out, label, (x1 + 3, ly), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1,
                        cv2.LINE_AA)
        return out

def open_camera(args: argparse.Namespace) -> Picamera2:
    tuning = Picamera2.load_tuning_file(str(args.tuning_file))
    camera = Picamera2(args.camera_num, tuning=tuning)
    controls = {"FrameRate": args.camera_fps}
    optional_controls = {
        "Sharpness": args.sharpness,
        "Contrast": args.contrast,
        "Saturation": args.saturation,
        "ExposureTime": args.shutter_us,
        "AnalogueGain": args.analogue_gain,
    }
    supported_controls = getattr(camera, "camera_controls", {}) or {}
    for key, value in optional_controls.items():
        if value is None:
            continue
        if supported_controls and key not in supported_controls:
            print(f"[camera] control {key} is not supported; skipping")
            continue
        controls[key] = value
    config = camera.create_video_configuration(
        main={"size": (args.width, args.height), "format": args.camera_format},
        controls=controls,
        buffer_count=4,
        queue=False,
    )
    camera.configure(config)
    camera.start()
    time.sleep(1.0)
    return camera

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Raspberry Pi 5 NCNN two-stage signboard preview")
    parser.add_argument(
        "--preset",
        choices=["realtime", "balanced", "full-coverage", "accuracy", "custom"],
        default="custom",
        help="realtime targets 10+ FPS; full-coverage runs overlapping tiles over 640x480 every frame",
    )
    parser.add_argument("--detector", default=str(ACCURACY_DETECTOR), help="Main-thread detector (runs every frame)")

```



```

parser.add_argument("--classifier", default=str(ACCURACY_CLASSIFIER), help="Crop classifier model")
parser.add_argument("--tile-detector", default=str(ACCURACY_DETECTOR), help="Tile-worker detector
(can be larger/slower, runs async)")
parser.add_argument("--tuning-file", type=Path, default=DEFAULT_TUNING)
parser.add_argument("--camera-num", type=int, default=0)
parser.add_argument("--width", type=int, default=1280)
parser.add_argument("--height", type=int, default=720)
parser.add_argument("--camera-fps", type=float, default=30.0)
parser.add_argument("--camera-format", default="RGB888",
                        help="Picamera2 main stream format. Keep RGB888 unless you have confirmed BGR888
8
works.")
parser.add_argument("--camera-color", choices=["rgb", "bgr"], default="bgr",
                        help="Color order returned by the camera stream. RGB frames are converted to BG
R
for YOLO/OpenCV. "
                        "On most RPi Picamera2 setups with RGB888, the V4L2 memory is actually BGR
,
so 'bgr' (no conversion) is correct.")
parser.add_argument("--camera-rotation", type=int, choices=[0, 90, 180, 270], default=0,
                        help="Rotate camera input image clockwise by this angle (0, 90, 180, or 270) to
normalise it to landscape")
parser.add_argument("--sharpness", type=float, default=None,
                        help="Optional libcamera Sharpness control, e.g. 1.3-1.6 for crisper sign
edges.")
parser.add_argument("--contrast", type=float, default=None,
                        help="Optional libcamera Contrast control.")
parser.add_argument("--saturation", type=float, default=None,
                        help="Optional libcamera Saturation control.")
parser.add_argument("--shutter-us", type=int, default=None,
                        help="Optional fixed exposure time in microseconds. Lower values reduce motion
blur but need more light.")
parser.add_argument("--analogue-gain", type=float, default=None,
                        help="Optional fixed analogue gain. Useful with --shutter-us in controlled
tests.")
parser.add_argument("--det-imgsz", type=int, default=640, help="Inference size for main-thread
detector")
parser.add_argument("--cls-imgsz", type=int, default=640, help="Inference size for classifier")
parser.add_argument("--tile-imgsz", type=int, default=640, help="Inference size for tile-worker
detector")
parser.add_argument("--tile-size", type=int, default=480)
parser.add_argument("--tile-overlap", type=float, default=0.30)
parser.add_argument("--tile-roi", choices=["full", "upper", "horizon", "center"], default="full",
                        help="Region used by rolling tile worker; full preserves maximum coverage")
parser.add_argument("--tile-scan-order", choices=["center", "scan"], default="center",
                        help="center refreshes central camera tiles first; scan keeps simple left-to-
right order")
parser.add_argument("--tile-priority-every", type=int, default=0,
                        help="Refresh the center tile every N tile cycles. 0 disables repeated center
priority.")
parser.add_argument("--tile-budget", type=int, default=1,
                        help="Number of tiles to refresh per tile-worker cycle. Higher refreshes covera
ge
faster but costs CPU.")

```

```

parser.add_argument("--tile-cache-ttl", type=float, default=0.75,
                    help="Seconds to keep per-tile detections while the rolling sweep refreshes.")
parser.add_argument("--tile-clahe", action="store_true", help="Enable tile CLAHE; improves some low
-
light scenes but costs FPS")
parser.add_argument("--tile-min-box", type=int, default=6)
parser.add_argument("--det-conf", type=float, default=0.25)
parser.add_argument("--det-iou", type=float, default=0.55)
parser.add_argument("--merge-iou", type=float, default=0.40)
parser.add_argument("--cls-conf", type=float, default=0.80)
parser.add_argument("--reject-class", default="not_target")
parser.add_argument("--max-det", type=int, default=20)
parser.add_argument("--classify-every", type=int, default=3)
parser.add_argument("--tracker", default="bytetrack.yaml", help="Legacy option; runtime uses the
built-in lightweight tracker")
parser.add_argument("--no-track", action="store_true", help="Legacy option retained for CLI
compatibility")
parser.add_argument("--audio-debounce", type=float, default=0.25, help="Time in seconds of continuo
us
detection before audio is voiced")
parser.add_argument("--language", choices=["english", "tamil"], default="english", help="Voice
announcement language")
parser.add_argument("--crop-pad", type=float, default=0.18)
parser.add_argument("--min-crop-side", type=int, default=48)
parser.add_argument("--track-ttl", type=int, default=12)
parser.add_argument("--track-ttl-sec", type=float, default=0.75,
                    help="Seconds before a missing sign track is discarded.")
parser.add_argument("--track-iou", type=float, default=0.25)
parser.add_argument("--track-center-frac", type=float, default=0.14,
                    help="Center-distance fallback for matching fast moving signs across frames.")
parser.add_argument("--main-every", type=int, default=5,
                    help="Run full-frame detector every N display frames. 0 disables full-frame
detector.")
parser.add_argument("--full-cache-ttl", type=float, default=0.45,
                    help="(Legacy, now ignored) Full-frame results persist until the next refresh
cycle.")
parser.add_argument("--result-ttl", type=float, default=1.5,
                    help="Seconds after which inference results are marked stale (dimmed). Detectio
ns
still display until --persist-ttl.")
parser.add_argument("--persist-ttl", type=float, default=3.0,
                    help="Seconds to keep showing last-known detections on screen between inference
cycles. "
                    "Should be longer than --result-ttl. 0 disables persistence (old flickerin
g
behaviour).")
parser.add_argument("--threads", type=int, default=4)
parser.add_argument("--zoom-mode", choices=["off", "tiles", "hybrid"], default="hybrid",
                    help="Tile inference mode: off (no tiles), tiles (tiles in background), hybrid
(both)")
parser.add_argument("--tile-conf", type=float, default=0.15, help="Confidence threshold for tile
detector")
parser.add_argument("--min-box-frac", type=float, default=0.012,

```

```

        help="Min detection width as fraction of frame width. Rejects very distant (tiny)
signs.")
    parser.add_argument("--max-box-frac", type=float, default=0.55,
        help="Max detection width as fraction of frame width. Rejects very close / huge
false positives.")
    parser.add_argument("--max-aspect-ratio", type=float, default=6.0,
        help="Max allowed aspect ratio (longer_side / shorter_side). Rejects extreme
slivers.")
    parser.add_argument("--audio-stability", type=int, default=3,
        help="Minimum fresh detector+classifier confirmations before audio can fire.")
    parser.add_argument("--audio-reclassify-fresh", action=argparse.BooleanOptionalAction, default=True,
        help="Re-run classifier for fresh detections until audio is confirmed.")
    parser.add_argument("--audio-cls-gate", type=float, default=0.85,
        help="Minimum classifier confidence required for audio announcement.")
    parser.add_argument("--ir-gpio", type=int, default=None, help="GPIO pin to control IR sensors / LEDs
(e.g. 18)")
    parser.add_argument("--display", choices=["window", "none"], default="window")
    parser.add_argument("--preview-width", type=int, default=800, help="Scale preview window width to
this size; set to 0 or less to disable scaling")
    parser.add_argument("--preview-height", type=int, default=None, help="Scale preview window height to
this size; defaults to maintaining aspect ratio")
    parser.add_argument("--benchmark-frames", type=int, default=0, help="Exit after N processed frames;
useful with --display none")
    args = parser.parse_args()

    raw_argv = sys.argv[1:]

    def provided(option: str) -> bool:
        return any(arg == option or arg.startswith(option + "=") for arg in raw_argv)

    def preset_default(attr: str, value, option: str) -> None:
        if not provided(option):
            setattr(args, attr, value)

    if args.preset == "realtime":
        preset_default("detector", str(FAST_DETECTOR), "--detector")
        preset_default("classifier", str(FAST_CLASSIFIER), "--classifier")
        preset_default("tile_detector", str(FAST_DETECTOR), "--tile-detector")
        preset_default("det_imgsz", 256, "--det-imgsz")
        preset_default("cls_imgsz", 256, "--cls-imgsz")
        preset_default("tile_imgsz", 256, "--tile-imgsz")
        preset_default("tile_budget", 1, "--tile-budget")
        preset_default("main_every", 8, "--main-every")
        preset_default("result_ttl", 1.2, "--result-ttl")
        preset_default("persist_ttl", 2.5, "--persist-ttl")
        preset_default("tile_cache_ttl", 0.55, "--tile-cache-ttl")
    elif args.preset == "balanced":
        preset_default("detector", str(BALANCED_DETECTOR), "--detector")
        preset_default("classifier", str(BALANCED_CLASSIFIER), "--classifier")
        preset_default("tile_detector", str(BALANCED_DETECTOR), "--tile-detector")

```

```

        preset_default("det_imgsz", 320, "--det-imgsz")
        preset_default("cls_imgsz", 320, "--cls-imgsz")
        preset_default("tile_imgsz", 320, "--tile-imgsz")
        preset_default("tile_budget", 1, "--tile-budget")
        preset_default("main_every", 6, "--main-every")
        preset_default("result_ttl", 1.5, "--result-ttl")
        preset_default("persist_ttl", 3.0, "--persist-ttl")
        preset_default("tile_cache_ttl", 0.65, "--tile-cache-ttl")
    elif args.preset == "full-coverage":
        preset_default("detector", str(BALANCED_DETECTOR), "--detector")
        preset_default("classifier", str(BALANCED_CLASSIFIER), "--classifier")
        preset_default("tile_detector", str(BALANCED_DETECTOR), "--tile-detector")
        preset_default("width", 640, "--width")
        preset_default("height", 480, "--height")
        preset_default("det_imgsz", 320, "--det-imgsz")
        preset_default("cls_imgsz", 320, "--cls-imgsz")
        preset_default("tile_imgsz", 320, "--tile-imgsz")
        preset_default("tile_size", 320, "--tile-size")
        preset_default("tile_overlap", 0.20, "--tile-overlap")
        preset_default("tile_budget", 2, "--tile-budget")
        preset_default("main_every", 6, "--main-every")
        preset_default("result_ttl", 1.5, "--result-ttl")
        preset_default("persist_ttl", 3.0, "--persist-ttl")
        preset_default("tile_cache_ttl", 0.65, "--tile-cache-ttl")
    elif args.preset == "accuracy":
        preset_default("detector", str(ACCURACY_DETECTOR), "--detector")
        preset_default("classifier", str(ACCURACY_CLASSIFIER), "--classifier")
        preset_default("tile_detector", str(ACCURACY_DETECTOR), "--tile-detector")
        preset_default("det_imgsz", 640, "--det-imgsz")
        preset_default("cls_imgsz", 640, "--cls-imgsz")
        preset_default("tile_imgsz", 640, "--tile-imgsz")
        preset_default("tile_size", 320, "--tile-size")
        preset_default("tile_overlap", 0.20, "--tile-overlap")
        preset_default("tile_budget", 1, "--tile-budget")
        preset_default("main_every", 8, "--main-every")
        preset_default("result_ttl", 2.0, "--result-ttl")
        preset_default("persist_ttl", 4.0, "--persist-ttl")
        preset_default("tile_cache_ttl", 0.75, "--tile-cache-ttl")

    # Ensure imgsz matches preset model when preset overrides models
    # (presets already set det_imgsz/cls_imgsz above)

    return args

def main() -> int:
    args = parse_args()
    cv2.setNumThreads(args.threads)

    for model_path in (Path(args.detector), Path(args.classifier), Path(args.tile_detector),
args.tuning_file):
        if not model_path.exists():

```

```

        raise FileNotFoundError(model_path)

    detector = YOLO(args.detector, task="detect")
    classifier = YOLO(args.classifier, task="classify")
    camera = open_camera(args)
    state = RuntimeState()
    state.language = args.language
    last_infer_time = 0.0
    # Persistence: carry forward last good detections across display frames
    persisted_detections: list[TwoStageDetection] = []
    persisted_tile_count: int = 0
    persisted_time: float = 0.0 # monotonic time when persisted_detections were last refreshed
    infer_lock = threading.Lock()

    # Start background threads
    grabber = FrameGrabber(camera, rotation=args.camera_rotation, camera_color=args.camera_color)
    tile_worker = None
    if args.zoom_mode != "off":
        if Path(args.tile_detector).resolve() == Path(args.detector).resolve():
            tile_detector = detector
        else:
            tile_detector = YOLO(args.tile_detector, task="detect")
            tile_worker = TileWorker(tile_detector, args, infer_lock)
    infer_worker = InferenceWorker(detector, classifier, state, args, tile_worker, infer_lock)

    # Initialize IR GPIO if specified
    ir_led = None
    ir_state = "OFF"
    if args.ir_gpio is not None:
        if GPIO_SUPPORT:
            try:
                ir_led = LED(args.ir_gpio)
                ir_led.off() # shut off IR sensors/LEDs by default
                ir_state = "OFF"
                print(f"[IR] Initialized GPIO {args.ir_gpio} to OFF")
            except Exception as e:
                print(f"[IR] Error initializing GPIO {args.ir_gpio}: {e}")
        else:
            print("[IR] GPIO support (gpiozero) not available on this system")

    show_rejected = False
    paused = False
    fullscreen = False
    last_frame: np.ndarray | None = None
    preview_size: tuple[int, int] | None = None
    started = time.monotonic()
    fps_started = started
    frames = 0
    fps_frames = 0
    fps = 0.0

    print(f"Detector: {args.detector} (main, imsz={args.det_imsz})")
    print(f"Tile det: {args.tile_detector} (background, imsz={args.tile_imsz})")

```

```

print(f"Classifier: {args.classifier} (imgsz={args.cls_imgsz})")
print(f"Preset:      {args.preset}")
print(f"Camera:      Picamera2 #{args.camera_num} {args.width}x{args.height}@{args.camera_fps:g}")
print(
    f"Tiling:      zoom_mode={args.zoom_mode}, roi={args.tile_roi}, size={args.tile_size}, "
    f"overlap={args.tile_overlap:.2f}, budget={args.tile_budget},
    cache_ttl={args.tile_cache_ttl:.1f}s"
)
print(f"Full det:     every {args.main_every} frame(s), result_ttl={args.result_ttl:.1f}s,
persist_ttl={args.persist_ttl:.1f}s")
print("Controls:   [q] quit [p] pause [r] show/hide rejected [f] fullscreen [i] toggle IR")

try:
    while True:
        if not paused:
            frame = grabber.read()
            if frame is None:
                time.sleep(0.005)
                continue

            if tile_worker is not None:
                tile_worker.submit_frame(frame)
                infer_worker.submit_frame(frame)

            new_infer = False
            raw_detections, raw_tile_count, infer_time, infer_frame_time = infer_worker.get_latest(
)
            result_age = time.monotonic() - infer_frame_time if infer_frame_time > 0 else 999.0

            # Update persisted detections when we get fresh inference results
            if infer_time != last_infer_time and infer_time > 0:
                new_infer = True
                last_infer_time = infer_time
                if raw_detections: # new non-empty results → refresh persistence
                    persisted_detections = raw_detections
                    persisted_tile_count = raw_tile_count
                    persisted_time = time.monotonic()
                elif result_age <= args.result_ttl:
                    # Inference ran but found nothing on this frame → still trust it briefly
                    persisted_detections = []
                    persisted_tile_count = 0
                    persisted_time = time.monotonic()

            # Determine what to show: use persisted detections with TTL guard
            persist_age = time.monotonic() - persisted_time if persisted_time > 0 else 999.0
            persist_ttl = max(0.0, float(getattr(args, "persist_ttl", 3.0)))
            if persist_ttl > 0 and persist_age <= persist_ttl:
                detections = persisted_detections
                tile_count = persisted_tile_count
            else:
                detections = []
                tile_count = 0

```

```

if new_infer:
    # Track lifecycle & updates
    for det in detections:
        if det.accepted and det.track_id is not None and det.fresh:
            state.track_last_seen_frame[det.track_id] = state.frame_index
            if det.track_id not in state.track_first_seen:
                state.track_first_seen[det.track_id] = (time.monotonic(),
state.frame_index)

            track = state.object_tracks.get(det.track_id)
            if track is not None and det.classifier_fresh and
track.last_audio_confirm_frame != state.frame_index:
                if track.audio_class_name == det.classifier_name:
                    track.audio_confirmations += 1
                else:
                    track.audio_class_name = det.classifier_name
                    track.audio_confirmations = 1
                track.last_audio_confirm_frame = state.frame_index

    # Purge stale tracks
    stale_tracks = []
    for tid in list(state.track_first_seen.keys()):
        last_seen = state.track_last_seen_frame.get(tid, 0)
        if state.frame_index - last_seen > args.track_ttl:
            stale_tracks.append(tid)
    for tid in stale_tracks:
        state.object_tracks.pop(tid, None)
        state.track_first_seen.pop(tid, None)
        state.track_last_seen_frame.pop(tid, None)
        state.spoken_tracks.discard(tid)

    # Find candidates for audio trigger
    audio_stability = getattr(args, "audio_stability", 3)
    audio_cls_gate = getattr(args, "audio_cls_gate", 0.85)
    candidates = []
    for det in detections:
        if det.accepted and det.track_id is not None:
            if det.track_id not in state.spoken_tracks:
                if not det.fresh or not det.classifier_fresh:
                    continue
                track = state.object_tracks.get(det.track_id)
                if track is None:
                    continue
                if track.audio_class_name != det.classifier_name:
                    continue
                if track.audio_confirmations < audio_stability:
                    continue
                duration = time.monotonic() - track.first_seen_time
                if duration < args.audio_debounce:
                    continue
                if det.classifier_conf < audio_cls_gate:
                    continue
                priority = CLASS_PRIORITY.get(det.classifier_name, 3)
                candidates.append((priority, det))

```

```

        if candidates:
            # Sort candidates by priority tier (tier 1 is highest priority)
            candidates.sort(key=lambda x: x[0])
            best_priority, best_det = candidates[0]

            # Determine direction based on bounding box position in frame
            x1, y1, x2, y2 = best_det.box
            cx = (x1 + x2) * 0.5
            w = frame.shape[1]
            if cx < w / 3:
                direction = "left"
            elif cx > 2 * w / 3:
                direction = "right"
            else:
                direction = "ahead"

            # Class-specific direction fallbacks if ahead audio is not found
            fallback_direction = None
            if "left" in best_det.classifier_name.lower():
                fallback_direction = "left"
            elif "right" in best_det.classifier_name.lower():
                fallback_direction = "right"

            # Resolve path to the directory and audio file
            voice_dir = get_audio_dir(best_det.classifier_name)
            if voice_dir:
                audio_cache_key = (voice_dir, state.language, direction)
                if audio_cache_key in state.audio_file_cache:
                    audio_file = state.audio_file_cache[audio_cache_key]
                else:
                    audio_file = resolve_audio_file(voice_dir, state.language, direction)
                    state.audio_file_cache[audio_cache_key] = audio_file
                if not audio_file and direction == "ahead" and fallback_direction:
                    fallback_key = (voice_dir, state.language, fallback_direction)
                    if fallback_key in state.audio_file_cache:
                        audio_file = state.audio_file_cache[fallback_key]
                    else:
                        audio_file = resolve_audio_file(voice_dir, state.language,
fallback_direction)

                    state.audio_file_cache[fallback_key] = audio_file

            if audio_file:
                # Check if audio is currently playing
                is_playing = False
                if state.audio_process is not None:
                    if state.audio_process.poll() is None:
                        is_playing = True
                    else:
                        state.audio_process = None
                        state.current_playing_priority = 999

                play_new = False

```



```

        if not is_playing:
            play_new = True
        else:
            # Preempt lower-priority (higher number) audio
            if best_priority < state.current_playing_priority:
                try:
                    state.audio_process.terminate()
                    state.audio_process.wait(timeout=0.2)
                except Exception:
                    try:
                        state.audio_process.kill()
                    except Exception:
                        pass
                    state.audio_process = None
                    play_new = True

            if play_new:
                try:
                    state.audio_process = subprocess.Popen(
                        ["ffmpeg", "-nodisp", "-autoexit", "-loglevel", "quiet",
                        "-nostats", "-vn", audio_file],
                        stdout=subprocess.DEVNULL,
                        stderr=subprocess.DEVNULL
                    )
                    state.current_playing_priority = best_priority
                    state.spoken_tracks.add(best_det.track_id)
                    print(f"[audio] Playing: {best_det.classifier_name} {direction}
({state.language})")
                except Exception as e:
                    print(f"[audio] Error playing {audio_file}: {e}")

        frames += 1
        fps_frames += 1
        now = time.monotonic()
        if now - fps_started >= 1.0:
            fps = fps_frames / (now - fps_started)
            fps_frames = 0
            fps_started = now
        accepted = sum(det.accepted for det in detections)

        zoom_info = f" ({args.zoom_mode})" if args.zoom_mode != "hybrid" else ""
        infer_fps_info = f" infer:{infer_worker.infer_fps:.1f}Hz" if infer_worker.infer_fps >
0 else ""
        tile_fps_info = f"@{tile_worker.tile_fps:.1f}Hz" if tile_worker is not None and
tile_worker.tile_fps > 0 else ""
        if args.display == "window":
            if preview_size is None:
                pw = args.preview_width
                ph = args.preview_height
                h, w = frame.shape[:2]
                if (pw is not None and pw > 0) or (ph is not None and ph > 0):
                    if pw is not None and pw > 0 and ph is None:
                        ph = int(h * (pw / w))

```

```

        elif ph is not None and ph > 0 and pw is None:
            pw = int(w * (ph / h))
            preview_size = (pw, ph)
        else:
            preview_size = (w, h)
        if preview_size != (frame.shape[1], frame.shape[0]):
            preview_frame = cv2.resize(frame, preview_size, interpolation=cv2.INTER_AREA)
            scale_x = preview_size[0] / frame.shape[1]
            scale_y = preview_size[1] / frame.shape[0]
            last_frame = draw_detections(
                preview_frame,
                detections,
                show_rejected,
                scale_x=scale_x,
                scale_y=scale_y,
                copy_frame=False,
            )
        else:
            last_frame = draw_detections(frame, detections, show_rejected)

        cv2.putText(last_frame, f"FPS: {fps:.1f}{infer_fps_info} tiles:
{tile_count}{zoom_info}{tile_fps_info} full/{args.main_every}", (10, 26), cv2.FONT_HERSHEY_SIMPLEX,
0.65, (0, 255, 0), 2)
        cv2.putText(last_frame, f"accepted: {accepted} / proposals: {len(detections)}", (10
,
52), cv2.FONT_HERSHEY_SIMPLEX, 0.50, (255, 255, 255), 1)
        if args.ir_gpio is not None:
            cv2.putText(last_frame, f"IR sensor: {ir_state}", (10, 78),
cv2.FONT_HERSHEY_SIMPLEX, 0.50, (0, 0, 255) if ir_state == "ON" else (150, 150, 150), 1, cv2.LINE_AA
A)
            cv2.putText(last_frame, f"Language: {state.language.upper()}", (10, 104 if
args.ir_gpio is not None else 78), cv2.FONT_HERSHEY_SIMPLEX, 0.50, (255, 255, 0), 1, cv2.LINE_AA)

    key = 255
    if args.display == "window" and last_frame is not None:
        cv2.imshow("HearSight Pi 5 NCNN Preview", last_frame)
        key = cv2.waitKey(1) & 0xFF
    if key == ord("q"):
        break
    if key == ord("p"):
        paused = not paused
    if key == ord("r"):
        show_rejected = not show_rejected
    if key == ord("i"):
        if ir_led is not None:
            ir_led.toggle()
            ir_state = "ON" if ir_led.value else "OFF"
            print(f"[IR] Toggled IR filter/LED state to: {ir_state}")
        else:
            print("[IR] No IR GPIO configured or GPIO support not loaded. Use --ir-gpio
<pin>")
    if key == ord("l"):
        state.language = "tamil" if state.language == "english" else "english"

```

```

        print(f"[language] Switched language to: {state.language.upper()}")
    if key == ord("f") and args.display == "window":
        fullscreen = not fullscreen
        mode = cv2.WINDOW_FULLSCREEN if fullscreen else cv2.WINDOW_NORMAL
        cv2.setWindowProperty("HearSight Pi 5 NCNN Preview", cv2.WND_PROP_FULLSCREEN, mode)
    if args.benchmark_frames and frames >= args.benchmark_frames:
        break
finally:
    if state.audio_process is not None:
        try:
            state.audio_process.terminate()
        except Exception:
            pass
    if ir_led is not None:
        try:
            ir_led.close()
        except Exception:
            pass
    infer_worker.stop()
    if tile_worker is not None:
        tile_worker.stop()
    grabber.release()
    camera.stop()
    camera.close()
    cv2.destroyAllWindows()

    elapsed = time.monotonic() - started
    print(f"[done] processed {frames} frames in {elapsed:.2f}s ({frames / max(elapsed, 0.001):.2f} FPS)")
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

```

9. Detailed File Documentation: start_hearsight.sh

File location: testing/start_hearsight.sh

File size: 53 lines | **Language:** Bash (Shell Script)

9.1 Section-by-Section Explanation

Line 1 — Shebang

`#!/bin/bash` declares the interpreter. Bash is required for brace expansion (`{1..10}`), `BASH_SOURCE`, and other Bash-specific syntax.

Line 2 — Why 'set -e' is NOT used

Bluetooth commands (`bluetoothctl power on`, `bluetoothctl connect`) frequently fail during early boot if the BT adapter is not yet fully initialised. 'set -e' would terminate the entire script on any BT failure, preventing the Python detection script from running. Without 'set -e', failures are handled gracefully with '|| echo ...' fallbacks.

Lines 4–6 — Directory Resolution and Repo Root Change

`SCRIPT_DIR` uses `BASH_SOURCE[0]` (not `$0`) to find the script's own directory reliably, even if sourced or called via a symlink. `REPO_ROOT` navigates one level up (`testing/` → `repo root`). 'cd `REPO_ROOT`' ensures all relative paths in the Python script (`weights/`, `voice_recording/`) resolve correctly.

Line 9 — Boot Delay (sleep 5)

A 5-second delay allows the Raspberry Pi's display server, Picamera2/libcamera daemon, and Bluetooth hardware to fully initialise after login.

Line 12 — Bluetooth Power-On

'`bluetoothctl power on`' ensures the BT adapter is active. '|| echo ...' ensures the script continues even if the BT adapter is not ready. Non-fatal step.

Lines 15–29 — Background Bluetooth Connection

The BT connection block runs in a subshell '(...) &' so it executes asynchronously. The Python script starts immediately without waiting for BT to connect (good for wired earphone users). The loop retries `bluetoothctl connect` up to 10 times with 2-second pauses. Checks 'Connected: yes' before each attempt. Both Bluetooth and wired audio output are supported.

Line 32 — Virtual Environment Activation

'`source .venv-rpi/bin/activate`' activates the Python venv, ensuring `picamera2`, `ultralytics`, `opencv-python` etc. from the venv are used instead of system packages.

Lines 33–52 — Python Script Launch and Runtime Arguments

The Python detection script is launched with the full path from `SCRIPT_DIR`. Backslash-continued command passes all production runtime arguments (see Section 9.2).

9.2 Meaning of Each Runtime Argument

Argument	Default	Meaning
--preset accuracy	accuracy	640px NCNN accuracy preset; uses best_ncnn_model_640 for all models.
--display window	window	Enables the OpenCV preview window.
--camera-rotation 270	0	Rotate frame 270° clockwise (90° CCW) to correct camera mounting.
--sharpness 1.4	None	libcamera Sharpness control — crisper sign edges.
--contrast 1.08	None	libcamera Contrast control — slightly enhanced contrast.
--threads 3	4	Limit OMP/OpenBLAS threads to 3 for system balance.
--main-every 1	5 (preset)	Run full-frame detector every frame (override preset default of 8).
--result-ttl 2.0	2.0	Results older than 2s are considered stale.
--persist-ttl 4.0	4.0	Last-known detections remain on screen for up to 4 seconds.
--tile-budget 1	1	Process 1 tile per tile-worker cycle — keeps CPU cost low.
--tile-cache-ttl 0.65	0.75	Per-tile detection cache expires after 0.65s.
--tile-scan-order center	center	Refresh central frame tiles first (where signs most often appear).
--tile-priority-every 4	0	Repeat the highest-priority (center) tile every 4 tile cycles.
--det-conf 0.30	0.25	Minimum detector confidence. 0.30 reduces spurious proposals.
--tile-conf 0.20	0.15	Minimum tile detector confidence. 0.20 is stricter than default.
--cls-conf 0.85	0.80	Minimum classifier confidence for a detection to be accepted.
--audio-cls-gate 0.7	0.85	Minimum classifier confidence gate before audio is played.
--audio-stability 1	3	Only 1 confirmed classification required before audio fires.
--audio-debounce 0.10	0.25	Sign must be detected for ≥0.10s before audio fires.

9.3 Full Source Code — start_hearsight.sh

```
#!/bin/bash
# Do NOT use 'set -e' — bluetooth commands may fail early at boot and must not abort the script.

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
cd "${REPO_ROOT}"

# Small delay to let display, camera, and bluetooth services initialise after login
sleep 5

# Ensure bluetooth is powered on (non-fatal if BT adapter not ready)
bluetoothctl power on || echo "[bluetooth] Failed to power on — continuing without BT"

# Attempt to connect to soundcore R50i VI in the background so it doesn't block startup
(
```

```
echo "Connecting to Soundcore R50i VI (18:9C:2C:4E:46:C8) in the background..."
for i in {1..10}; do
    # Check if already connected
    if bluetoothctl info 18:9C:2C:4E:46:C8 | grep -q "Connected: yes"; then
        echo "[bluetooth] Soundcore R50i VI is connected!"
        break
    fi
    if bluetoothctl connect 18:9C:2C:4E:46:C8; then
        echo "[bluetooth] Successfully connected to Soundcore R50i VI!"
        break
    fi
    sleep 2
done
) &

# Activate virtualenv and run hearsight (starts instantly for wired earphones / overall
# flow)
source .venv-rpi/bin/activate
python "${SCRIPT_DIR}/rpi_webcam_test_two_stage_ncnn.py" \
    --preset accuracy \
    --display window \
    --camera-rotation 270 \
    --sharpness 1.4 \
    --contrast 1.08 \
    --threads 3 \
    --main-every 1 \
    --result-ttl 2.0 \
    --persist-ttl 4.0 \
    --tile-budget 1 \
    --tile-cache-ttl 0.65 \
    --tile-scan-order center \
    --tile-priority-every 4 \
    --det-conf 0.30 \
    --tile-conf 0.20 \
    --cls-conf 0.85 \
    --audio-cls-gate 0.7 \
    --audio-stability 1 \
    --audio-debounce 0.10
```

10. Complete Logic Explanation

Input — Camera Frame

Picamera2 captures frames from the RPi Camera Module via the 22-pin CSI connector. FrameGrabber reads frames continuously and applies rotation. Main loop reads without blocking.

Processing Stage 1 — Full-Frame Detector

YOLOv8 NCNN detector runs on the complete frame every `--main-every` frames, returning bounding boxes, confidence scores, and class IDs. Results cached until next refresh.

Processing Stage 2 — Rolling Tile Detector

TileWorker divides the frame into overlapping tile crops (`iter_tiles`). Each tile is processed with the detector. Per-tile results cached by coordinates. Catches small/distant signs missed by the full-frame pass.

Filtering — Confidence, IoU, NMS, and Gates

Detector confidence filters in YOLO predict (`--det-conf`, `--tile-conf`). NMS within predict (`--det-iou`). Class-aware merge NMS combines full-frame + tile at `--merge-iou` with full-frame priority. Depth/size gate rejects by `--min-box-frac`, `--max-box-frac`, `--max-aspect-ratio`.

Classification — Two-Stage Pipeline

Each surviving detection is cropped (`crop_xyxy`). YOLOv8 NCNN classifier applied to crop. Top-1 class + confidence stored in `cls_cache`. Accepted if class `!= 'not_target'` and confidence `>= --cls-conf`.

Tracking — Stable IDs and Track Memory

`assign_track()` matches detections to existing tracks using composite score ($\text{IoU} \times 3 + \text{center proximity} + \text{hit bonus}$). New track assigned if no match. Stale tracks purged after `--track-ttl-sec`.

Output — Visual Bounding Boxes

`draw_detections()` renders green (accepted) / grey (rejected) boxes with filled label backgrounds. HUD shows FPS, inference rate, tile count, accepted/total, IR state, language.

Output — Audio Guidance

When track meets stability + debounce + confidence gates, `ffplay` launched as subprocess to play matching voice recording. File resolved by `voice_recording/<sign>/<lang>/<dir>.mp4`. Track added to `spoken_tracks` to prevent repeated announcements.

Direction Decision — Left / Ahead / Right

$\text{center_x} < \text{frame_width}/3 \rightarrow \text{left}$. $\text{center_x} > 2 \times \text{frame_width}/3 \rightarrow \text{right}$. Otherwise $\rightarrow \text{ahead}$. Class-name fallback (e.g., 'Side road left') used if 'ahead' file not found.

Language Selection

Active language stored in `RuntimeState`. `resolve_language_dir_and_files()` looks for language subdirs (`eng/tam/english/tamil`), falls back to filename keyword matching. [I] key switches language at runtime without restart.

Priority Handling

Tier 1 (critical/danger): Stop, No Entry, Danger Electricity, No Parking, etc. Tier 2 (warning): School Ahead, Railway Crossing, Speed Breaker, Cross Road, etc. Tier 3 (informational): Bus Stop, Hospital, Public Toilet. Tier 1 preempts any playing audio by terminating ffmpeg immediately.

11. Runtime Arguments Table

All command-line arguments accepted by `rpi_webcam_test_two_stage_ncnn.py`.

Argument	Default	Purpose	Example
--preset	custom	Performance preset: realtime, balanced, full-coverage, accuracy, custom.	--preset accuracy
--camera-num	0	Picamera2 camera index.	--camera-num 0
--width / --height	1280 × 720	Camera capture resolution in pixels.	--width 1280 --height 720
--camera-fps	30.0	Requested camera frame rate.	--camera-fps 30
--camera-format	RGB888	Picamera2 stream pixel format.	--camera-format RGB888
--camera-color	bgr	Colour order: 'rgb' applies BGR conversion for OpenCV/YOLO.	--camera-color bgr
--camera-rotation	0	Rotate frame clockwise: 0, 90, 180, or 270 degrees.	--camera-rotation 270
--tuning-file	ov5647_noi r.json	Path to libcamera tuning JSON for the camera sensor.	--tuning-file /path/to/file.json
--sharpness	None	libcamera Sharpness control value.	--sharpness 1.4
--contrast	None	libcamera Contrast control value.	--contrast 1.08
--saturation	None	libcamera Saturation control value.	--saturation 1.0
--shutter-us	None	Fixed exposure time in microseconds.	--shutter-us 5000
--analogue-gain	None	Fixed analogue gain.	--analogue-gain 4.0
--detector	ACCURAC Y path	Path to the NCNN detector model folder.	--detector weights/ detector/...
--classifier	ACCURAC Y path	Path to the NCNN classifier model folder.	--classifier weights /classifier/...
--tile-detector	ACCURAC Y path	NCNN model for tile background worker.	--tile-detector weig hts/detector/...
--det-imsz	640	Inference image size (px) for main detector.	--det-imsz 640
--cls-imsz	640	Inference image size (px) for classifier.	--cls-imsz 640
--tile-imsz	640	Inference image size (px) for tile detector.	--tile-imsz 640
--tile-size	480	Tile crop size in pixels.	--tile-size 320
--tile-overlap	0.30	Fractional overlap between adjacent tiles.	--tile-overlap 0.20
--tile-roi	full	ROI for tiling: full, upper, horizon, center.	--tile-roi full
--tile-scan-order	center	Tile order: center refreshes central tiles first; scan = left-to-right.	--tile-scan-order center
--tile-budget	1	Tiles processed per tile-worker cycle.	--tile-budget 1

Argument	Default	Purpose	Example
--tile-cache-ttl	0.75	Seconds to keep per-tile detections in cache.	--tile-cache-ttl 0.65
--tile-clahe	False	Enable CLAHE contrast enhancement for tile crops.	--tile-clahe
--tile-min-box	6	Minimum bounding box size in pixels within a tile.	--tile-min-box 6
--tile-priority-every	0	Re-process center tile every N tile cycles. 0 = disabled.	--tile-priority-every 4
--det-conf	0.25	Minimum detector confidence threshold.	--det-conf 0.30
--det-iou	0.55	IoU threshold for YOLO NMS within detector.	--det-iou 0.55
--merge-iou	0.40	IoU threshold for class-aware NMS merge of full + tile detections.	--merge-iou 0.40
--max-det	20	Maximum detections per inference call.	--max-det 20
--min-box-frac	0.012	Min detection width as fraction of frame width (depth gate).	--min-box-frac 0.012
--max-box-frac	0.55	Max detection width as fraction of frame width (size gate).	--max-box-frac 0.55
--max-aspect-ratio	6.0	Max allowed aspect ratio. Rejects extreme slivers.	--max-aspect-ratio 6.0
--main-every	5	Run full-frame detector every N frames. 0 = disabled.	--main-every 1
--result-ttl	1.5	Seconds after which inference results are marked stale.	--result-ttl 2.0
--persist-ttl	3.0	Seconds to persist last-known detections on screen.	--persist-ttl 4.0
--zoom-mode	hybrid	Tile inference mode: off / tiles / hybrid.	--zoom-mode hybrid
--cls-conf	0.80	Minimum classifier confidence for detection to be accepted.	--cls-conf 0.85
--reject-class	not_target	Classifier class that means background / no sign.	--reject-class not_target
--classify-every	3	Re-classify a cached track every N frames.	--classify-every 3
--crop-pad	0.18	Fractional padding around detection crop before classification.	--crop-pad 0.18
--min-crop-side	48	Minimum crop side length in pixels.	--min-crop-side 48
--track-ttl	12	Frames before a missing track is discarded.	--track-ttl 12
--track-ttl-sec	0.75	Seconds before a missing track is discarded.	--track-ttl-sec 0.75
--track-iou	0.25	IoU gate for associating a detection to an existing track.	--track-iou 0.25
--track-center-frac	0.14	Center-distance fallback gate for fast-moving sign matching.	--track-center-frac 0.14
--language	english	Audio announcement language: english, tamil.	--language tamil
--audio-debounce	0.25	Seconds sign must be detected before audio fires.	--audio-debounce 0.10
--audio-stability	3	Minimum fresh detector+classifier confirmations before audio.	--audio-stability 1

Argument	Default	Purpose	Example
--audio-cls-gate	0.85	Minimum classifier confidence for audio announcement.	--audio-cls-gate 0.7
--display	window	Display mode: window shows preview; none for headless.	--display none
--preview-width	800	Scale preview window to this width (px). 0 = no scaling.	--preview-width 800
--preview-height	None	Scale preview window to this height. Default: maintain aspect ratio.	--preview-height 600
--benchmark-frames	0	Exit after N processed frames (for benchmarking).	--benchmark-frames 100
--threads	4	CPU threads for OMP/OpenBLAS and OpenCV.	--threads 3

12. Dependencies / Installation Notes

Commands below are example commands and may need adaptation to your environment.

Python Virtual Environment

```
python3 -m venv .venv-rpi
source .venv-rpi/bin/activate
```

Isolates project packages from system Python.

Install Python Packages

```
pip install -r requirements.txt
# Key: ultralytics>=8.0.0, opencv-python>=4.8.0, numpy>=1.24.0
```

Install inside the venv.

Picamera2 and libcamera

```
# Usually pre-installed on Raspberry Pi OS
sudo apt install python3-picamera2 libcamera-dev
```

Picamera2 must be at system level or linked into the venv.

FFmpeg (ffplay)

```
sudo apt install ffmpeg
```

Required for audio playback. ffplay must be in system PATH.

Bluetooth Tools

```
# Pre-installed on Raspberry Pi OS
sudo apt install bluez
```

Required for Bluetooth earphone connection in the shell script.

Model Files

```
# Ensure NCNN model folders exist:
# weights/detector/best_ncnn_model_640/
# weights/classifier/best_ncnn_model_640/
```

Folders must contain NCNN .param and .bin files from Ultralytics export.

Audio Files

```
# Structure:
# voice_recording///.mp4
```

Audio files organised by sign name, language, and direction.

13. Build / Run Instructions

13.1 Make the Shell Script Executable

```
chmod +x testing/start_hearsight.sh
```

13.2 Run via Startup Script (Recommended)

From the repository root:

```
./testing/start_hearsight.sh
```

Activates the venv, handles Bluetooth, and launches the detection script with all production arguments.

13.3 Manual Python Run

Activate the virtual environment, then run:

```
source .venv-rpi/bin/activate
python testing/rpi_webcam_test_two_stage_ncnn.py \
  --preset accuracy --display window --camera-rotation 270 \
  --sharpness 1.4 --contrast 1.08 --threads 3 --main-every 1 \
  --det-conf 0.30 --cls-conf 0.85 --audio-cls-gate 0.7 \
  --audio-stability 1 --audio-debounce 0.10
```

13.4 Keyboard Controls

Key	Action
[q]	Quit the application and cleanly shut down all threads
[p]	Pause / unpause the detection loop
[r]	Toggle display of rejected (grey) proposals
[f]	Toggle fullscreen mode
[l]	Switch audio language between English and Tamil